PDF-to-JSON Extractor — Solution Notes

Approach

After reading the task and looking at the raw MuPDF output from all three PDFs, I decided against reaching for an LLM immediately. The forms are California Judicial Council AcroForm PDFs — some of the most consistently structured documents that exist. Every fillable field already has a globally unique widget name baked into the PDF (e.g. `SC-100[0].Page2[0].List1[0].Item1[0].PlaintiffName1[0]`), and the target schema encodes those widget paths in its `mappings` array. That means extraction is fundamentally a **Map lookup**, not a parsing problem.

The real challenge is robustness: checkboxes don't always behave, some PDFs might have no text layer at all, and the task explicitly asks for handling of "various PDF documents" — not just the three samples. So I designed a tiered hybrid system:

Tier 1 — Deterministic widget extraction for known Judicial Council forms. Fast, O(1) per field, no API calls needed.

Tier 2 — Checkbox robustness via a three-step resolver: widget value first, then Unicode symbol scan (the `þ` glyph in SC-101 was a real signal here), then pixel density on a rendered greyscale image if the first two are inconclusive.

Tier 3 — LLM vision fallback, but only where it genuinely adds value: scanned PDFs with no text layer, fields where confidence drops below 0.7, or any PDF that doesn't match a known form prefix (routed through a schema-guided generic extractor). LLM is never called first — it's a targeted last resort, which keeps cost and latency down and makes the system auditable.

Architecture Overview

```
src/
  types.ts          Shared interfaces and ExtractionError class
  extract.ts        Layer 1  — MuPDF wrapper
  detect.ts         Layer 2  — Form type detection
  checkboxes.ts     Layer 4  — Checkbox resolver
  validator.ts      Layer 5  — Confidence scorer
  llm.ts            Layer 6  — LLM fallback (targeted, flagged fields only)
  extractors/
    sc100.ts        Layer 3  — SC-100 per-form extractor ┐
    sc101.ts        Layer 3  — SC-101 per-form extractor ├─ Path A
    sc103.ts        Layer 3  — SC-103 per-form extractor ┘
    generic.ts      Layer 3  — Schema-guided LLM extractor (Path B)
  index.ts          Layer 7  — Orchestrator
```

```
output/
  sc100.json, sc101.json, sc103.json
  extraction-report.json
```

Two-Path Extraction Architecture

The orchestrator (`src/index.ts`) routes each PDF through one of two paths depending on whether the form type was recognised.

Path A — Deterministic

```
extractPDF → detectFormType
  → per-form extractor (widget Map lookup + MCQ resolution)
  → confidence scorer
  → targeted LLM fallback for flagged fields only
  → write output JSON
```

Used for: SC-100, SC-101, SC-103 AcroForm PDFs where every fillable field has a known widget name.

- Widget names are looked up directly in `widgetsByName` — O(1), no text parsing.
- LLM is called only if confidence < 0.7 or more than 3 fields are flagged, and only for those specific fields.
- Deterministic output — same form always produces the same JSON.

Path B — Generic LLM (unknown / arbitrary PDFs)

```
extractPDF → detectFormType (returns 'unknown')
  → extractGeneric (schema-guided Gemini Vision)
    → collectLeafFields: walk sc100 schema for all 83 extractable questions
    → for each page: render PNG → queryPage → Gemini answers all questions
    → first non-null answer per question wins across pages
    → write answers back onto cloned schema nodes
  → confidence scorer
  → write output JSON
```

Used for: any PDF that does not match a known form prefix or text anchor. This includes arbitrary court documents, scanned forms, or future form versions not yet in the schema.

- No bounding-box assumptions — the LLM reads the image directly.
- Every page is sent to Gemini with the full question catalogue.
- Output shape is identical to Path A so the same confidence scorer and JSON writer apply.

- Token cost is higher (whole-form extraction per page) — acceptable because Path B is only invoked when deterministic routing fails.

Routing decision

```typescript
const formType = detectFormType(extraction); // 'SC-100' | 'SC-101' | 'SC-103' | 'unknown'

if (formType === 'unknown') {
  // Path B: full LLM extraction
  const result = await extractGeneric(pdfPath);
  ...
  return; // done
}

// Path A: deterministic + optional targeted LLM fallback
```

`detectFormType` never throws — it returns `'unknown'` and lets the orchestrator decide what to do next.

---

1. Why Tiered Extraction (Deterministic First, LLM Last)

The problem with LLM-first

Reaching for a vision model immediately felt like the wrong call here. LLM calls are slow (2–10s per round trip), non-deterministic (same form, different output), rate-limited at scale, and expensive on vision tokens. More importantly, for these forms it's unnecessary.

The deterministic advantage for these forms

California Judicial Council forms are published as interactive AcroForm PDFs. Every fillable field has a globally unique widget name baked into the PDF structure (e.g. `SC-100[0].Page2[0].List1[0].Item1[0].PlaintiffName1[0]`).

The target schema (`src/sc100.ts`) already encodes these widget paths in the `mappings` array of every field. This means extraction reduces to a **Map lookup** — no text parsing required for filled forms.

The cascade

```
Widget lookup (O(1), most reliable)
  → if missing: Anchor-based text proximity search
    → if missing: LLM vision API for flagged fields only
```

LLM is invoked only when:

- The PDF lacks a machine-readable text layer (scanned), OR
- Overall confidence drops below 70%, OR
- More than 3 fields are unresolved

Even then, the LLM is only asked about the specific flagged questions — never the whole form. This keeps token usage proportional to what is actually missing.

---

2. Anchor-Based Extraction and Its Limits

How it works

Each form has printed label text such as `"Name:"`, `"Phone:"`, `"Street address:"`. The extractor:

1. Finds the label line whose text contains the label substring
2. Looks for a candidate text line that is either:
    - On the **same row** (within ±6 pt vertically) and to the **right** of the label (within 200 pt)
    - **Below** the label (within 40 pt) at roughly the same left margin

The bounding boxes come from MuPDF's `StructuredText` JSON output, which includes per-line `bbox` objects with `x`, `y`, `w`, `h` in PDF points.

Why widgets win

Anchor matching works well for single-value fields next to a clear label. It degrades when:

- The label spans multiple lines or wraps
- The form uses floating label-above-input layout (common in these forms)
- Two fields share a row (e.g. City / State / Zip on the same line)
- The filled value itself contains characters that look like label text

For the Judicial Council AcroForm PDFs, widget extraction is always preferred because the widget coordinate/name data is authoritative and structurally reliable.

Known anchor limits

Scenario	Outcome
Multi-line label	Only first line matched; may miss context

Scenario	Outcome
Value on next line, indented	Caught by the "below within 40 pt" heuristic, but fragile
Two fields on same horizontal line	First match wins; second may be missed
Scanned image overlay over text layer	Widget values empty; anchor gets OCR artefacts

3. Checkbox Resolver

Checkboxes are resolved in three steps (`src/checkboxes.ts`):

Step 1 — Widget value

AcroForm checkboxes export their state as a string. The PDF specification says:

- `"Off"` → unchecked
- Any other value (typically `"Yes"`, `"On"`, or a custom export value) → checked

The resolver checks whether the widget value belongs to the known `UNCHECKED_VALUES` set. If not (and the value is non-empty), the checkbox is checked.

```
resolveCheckboxFast(widget, []) → { checked: boolean, method: 'widget' }
```

Step 2 — Symbol scan

Some PDFs embed checkbox state as Unicode glyphs in the text layer rather than in an AcroForm value. The `þ` character appearing in SC-101's raw text output was the concrete signal that led to this step — it's a custom font glyph rendering a checkmark. The resolver scans nearby text lines for any character in the set:

```
{ 'þ', 'ý', '✓', '✔', '☑', '■', '●', '•', 'x', 'X' }
```

Step 3 — Pixel density

When neither widget nor symbol yields a definitive answer (scanned forms, graphic-only checkboxes), the page is rendered to a greyscale pixmap at 3× resolution using MuPDF's `toPixmap`. The checkbox bounding box is mapped to pixel coordinates and the dark pixel ratio (luminance < 128) is computed. If > 15% of the pixels inside the box are dark, the checkbox is considered checked.

This is the most expensive step and is skipped for AcroForm PDFs where step 1 is conclusive.

```
resolveCheckbox(widget, nearbyText, filePath, pageIndex, bbox)
  → { checked: boolean, method: 'widget' | 'symbol' | 'pixel' | 'unknown' }
```

---

4. Error Handling

The system uses a typed `ExtractionError` class with distinct error codes so failures are never ambiguous:

Error code	When it fires	Behaviour
`FormTypeUnknown`	Form prefix not recognised, no text anchors match	Routes to Path B (generic LLM extractor)
`NoTextLayer`	MuPDF finds 0 text lines across all pages	LLM fallback triggered unconditionally
`SchemaValidationFailed`	Extracted output doesn't conform to expected types	Logged with field-level diff; partial output still saved
`LLMFallbackFailed`	API call fails or returns unparseable JSON	Logged with original error; extraction continues with deterministic-only result

Single form failures are isolated inside a per-form `try/catch` in the orchestrator. If SC-101 fails, SC-100 and SC-103 still complete and their results are written. The failure is recorded in `extraction-report.json` with the error code and message.

The system never silently swallows errors — every failure path logs context (which PDF, which field, which step) so failures are reproducible and debuggable.

---

5. What Would Improve Accuracy with More Time

Filled-form test corpus

The current PDFs are blank Judicial Council templates. Without filled examples, all fields score 0.4 (present but empty). A set of intentionally filled PDFs — or synthetic fills via MuPDF's widget-set API — would let us validate the end-to-end pipeline and tune thresholds. SC-103 jumped to 75% confidence immediately when filled with test data, which confirms the extraction logic is correct.

MCQ resolution completeness

Multiple-choice parent fields (e.g. "Is the plaintiff mailing address different?") are inferred from associated checkbox widgets or from the presence of child field values. Mapping every MCQ to its exact radio button

widget pair would eliminate the remaining 0.4-confidence MCQ fields.

Anchor-based improvements

- Use the **centroid distance** between label and candidate instead of separate horizontal/vertical thresholds
- Account for **right-aligned labels** (common in header sections)
- Build a per-form coordinate map (label → expected value bbox) so anchor matching becomes a direct lookup rather than a proximity search

Confidence calibration

The current 1.0 / 0.7 / 0.4 / 0.0 tiers are coarse. A logistic regression calibrated on a labelled dataset would give more accurate per-field probability estimates.

LLM prompt improvement

The current prompt asks for all flagged fields at once. Structured output (JSON schema forcing) via the API would eliminate the JSON-parse-failure fallback path.

---

6. Known Edge Cases and Handling

Edge case	Detection	Handling
Blank / unfilled template	All widget values empty → confidence ≈ 0.4	Flagged → LLM fallback candidate; reports as "unfilled"
Scanned image (no text layer)	`hasTextLayer = false`	LLM fallback triggered unconditionally
Broken structure tree	MuPDF emits warning to stderr	Non-fatal; text extraction continues normally
Multi-part SC-103 / SC-101 (nested inside SC-100)	Detected via widget prefix in `detectFormType`	Standalone extractors provided; SC-100 extractor also walks nested document children in schema
Multiple defendants / plaintiffs	Widget names follow `[0]`/`[1]` suffix pattern	Both indexed widget pairs are extracted
More than one form per PDF	Not encountered in the provided files	`detectFormType` returns first match; would need multi-pass for combined PDFs
Amount fields with currency symbols	Regex strips non-numeric characters before `parseFloat`	If value is non-numeric after strip, original string is preserved with confidence 0.7

Edge case	Detection	Handling
Missing `GEMINI_API_KEY`	Checked before LLM call	Warning logged; extraction continues with deterministic results (Path A) or throws for Path B
Single form failure	Caught in per-form try/catch	Failure reported in `extraction-report.json`; other forms continue

Running

```bash
# Extract the three known forms (Path A — no API key needed)
npm run extract

# With LLM fallback enabled for low-confidence fields (Path A) or unknown forms (Path B)
GEMINI_API_KEY=... npm run extract

# Type check only
npm run typecheck
```

Output files are written to `output/`:

- `sc100.json` — Plaintiff's Claim (SC-100)
- `sc101.json` — Attorney Fee Dispute (SC-101)
- `sc103.json` — Fictitious Business Name (SC-103)
- `extraction-report.json` — Summary of all forms with confidence scores